PARIS PANTHÉON-ASSAS UNIVERSITÉ

ST2MLE : Machine Learning for IT Engineers

LAB 1 :

- **Exploratory data analysis (EDA)**
- **Data pre-processing**
- **Class-imbalance**
- **Supervised learning with decision trees, Naïve Bayes, Bagging, Boosting and Stacking, hyper-parameters fine-tuning, regularization**

Context: Software Defect Prediction

The **Software Defect Prediction dataset**, commonly referred to as **PC4 (**Project C, Release 4), is part of a collection of datasets used in empirical software engineering research, particularly for **predicting software bugs or defects** from static code metrics.

The **PC4** dataset is one of several datasets from the **NASA Metrics Data Program (MDP)**, collected to study the relationship between software metrics and defects in software modules. Each row represents a software module, and the features describe various **software complexity, size, and structure metrics**. The target variable indicates whether that module was **faulty (defective)** or not.

MDP = **Metrics Data Program**.

This was a program initiated by NASA (in collaboration with researchers at the University of Maryland and later others) to **collect and publicly release software metrics and defect data** from NASA missions. The goal was to promote **empirical studies in software engineering**, especially related to software quality, prediction models, and software process improvement.

Requirements

```
pip install pandas scikit-learn seaborn matplotlib imbalanced-learn openml
```

Dataset

We use the **PC4 dataset** from NASA's Metrics Data Program (MDP), among other available MDP datsets, due to its realistic structure, manageable size, and strong class imbalance (around 12% defective modules). These properties make PC4 particularly suitable for hands-on machine learning tasks such as exploratory data analysis, preprocessing, resampling with SMOTE, and evaluating the performance of various classifiers. Its balance between complexity and accessibility provides a robust foundation for model comparison and critical discussion of results.

You can download the PC4 dataset from the klainfo/NASADefectDataset repository

Also from Openml: https://www.openml.org/search?type=data&status=active&id=1049

Or use openml.datasets.get_dataset(id) # id is the id of the dataset PC4.

Part 1: Exploratory Data Analysis (EDA)

Objectives:

- Understand the feature set
- Understand data distributions
- Check class imbalance, missing values, and outliers

Tasks:

1. Load the dataset.
2. Create a summary table for the different features (Feature, Description)
3. Summary statistics with df.describe().
4. Visualize:
    - Histograms / KDE plots
    - Correlation heatmap
    - Class distribution (sns.countplot)
5. Check for:
    - Missing values (df.isnull().sum())
    - Outliers (e.g., box plots, IQR)
6. Normalize and standardize numeric features (if you think it is needed).
7. Summarize the different insights of the EDA phase: Are there issues with some features? High feature correlation ? Missing values ? Outliers ?
8. Take the necessary action(s) and justify.

Part 2: Train-Test Split and Baseline Model

1. Use a stratified train_test_split (80%:train-20%:test)
2. Train a **baseline Decision Tree**.
3. Report accuracy on both training and testing sets.
4. Report precision, recall, F1 and the confusion matrix on the testing set.
5. Any issue of overfitting? Underfitting? Other issue(s)?

Part 3: Resampling with SMOTE

Objectives:

- Understand class imbalance handling using SMOTE

Tasks:

1. Based on the class distribution, do we need under-sampling or oversampling? Justify your choice.
2. Apply SMOTE using the imblearn.over_sampling module.
3. Show class distribution **before** and **after**.
4. Is there any risk with SMOTE? Data leakage ?
5. Are there other oversampling methods?

Part 4: Classification Models (7 exercises)

Exercise 1: Decision Tree (baseline)

- Use DecisionTreeClassifier from sklearn.tree
- Report metrics (confusion matrix, precision, recall, F1)
- Analyze and discuss the results

Exercise 2: Decision Tree with Pre-Pruning

- Use GridSearchCV (with 5 folds) to tune max_depth, min_samples_split. Try with max_depth (3, 5, 10, 20, 30, None) and min_smaples_split (5, 10, 20, 30). Use “f1-macro” as the scoring metric for the grid search.
- Compare results with baseline

Exercise 3: Post-Pruning

- Use ccp_alpha pruning path and cost_complexity_pruning_path
- Compare results with baseline and with the pre-pruned tree

Exercise 4: Naive Bayes

- Use GaussianNB and/or MultinomialNB or (depending on feature preprocessing)
- Compare results with previous models

Exercise 5: Random Forest (Bagging)

- Use RandomForestClassifier (with n_estimators=100)
- Plot the feature importance and discuss the results
- Compare results with previous models

Exercise 6: Boosting (AdaBoost, GradientBoosting and XGBoost)

- Try AdaBoostClassifier (with n_estimators=100)
- GradientBoostingClassifier (with n_estimators=100)
- GradientBoostingClassifier (with n_estimators=100)
- Compare training time
- Compare results with previous models

Exercise 7: Stacking

- Use StackingClassifier with the previous top 3 performing models as base learners: (1) the pre-pruned decision tree with the optimal hyper-parameters, (2) the random forest and the (3) Gradient boosting. Use LogisticRegression as the meta learner.
- Evaluate performance

Part 5: Final Comparison & Discussion

- Create a comparison table of the different used classifiers:
    - Accuracy
    - Precision
    - Recall
    - F1-Score
- Use bar plots for visualization
- Reflect:
    - Which models handle imbalance better?
    - Which are more generalizable?
    - How does preprocessing (SMOTE) affect performance?

Deliverables (to be submitted on Moodle)

- A .zip file containing the following:
    - A Jupyter notebook or `.py` file with completed tasks
    - A pdf report including screenshots of the above experiments and a final comparison table with analyses.
    - 2 students can work together on the same lab and submit only one report.